EA2. Creación de repositorio

Caso de estudio: procesamiento funcional de datos para un sistema de gestión de transporte urbano

Sebastián Giraldo Anaya

Sara Gabriela Castro Mora

Institución Universitaria Digital de Antioquia

Programación Orientada a Objetos II

PREICA2602B020046

Jorge Armando Julio

5 de noviembre de 2026

TABLA DE CONTENIDO

1. Introducción 3
2. Objetivos 4
3. Desarrollo 5
    3.1 Modelo de datos 5
    3.2 Generador de datos simulados 9
    3.3 Tareas a, b y c: clase AnalizadorTransporte 10
    3.4 Tareas d, e y f: clase AnalisisAvanzado 13
    3.5 Clase principal Main 17
    3.6 Conceptos de programación funcional aplicados 20
    3.7 Evidencia de ejecución 21
    3.8 Repositorio y trabajo colaborativo 23
4. Conclusiones 24
5. Enlaces de entrega 25
6. Referencias 26

1. Introducción

En este trabajo desarrollamos en Java un módulo para procesar los datos de un sistema de transporte urbano, a partir del caso de estudio de la ciudad de TecnoValle. El sistema que tiene la ciudad hoy es lento y se equivoca en los conteos, porque procesa millones de registros uno por uno y con datos que se pueden modificar.

Para mejorarlo usamos programación funcional: funciones puras, colecciones que no se modifican, lambdas y Streams. Con eso calculamos la afluencia por estación, las horas pico, las rutas más usadas, los viajes de cada usuario, el tiempo entre estaciones y las rutas con sobrecarga. En el documento mostramos el código, explicamos cómo lo hicimos y dejamos las capturas de la ejecución.

2. Objetivos

Objetivo general

Aplicar la programación funcional en Java para procesar de forma declarativa una gran cantidad de datos de un sistema de transporte urbano.

Objetivos específicos

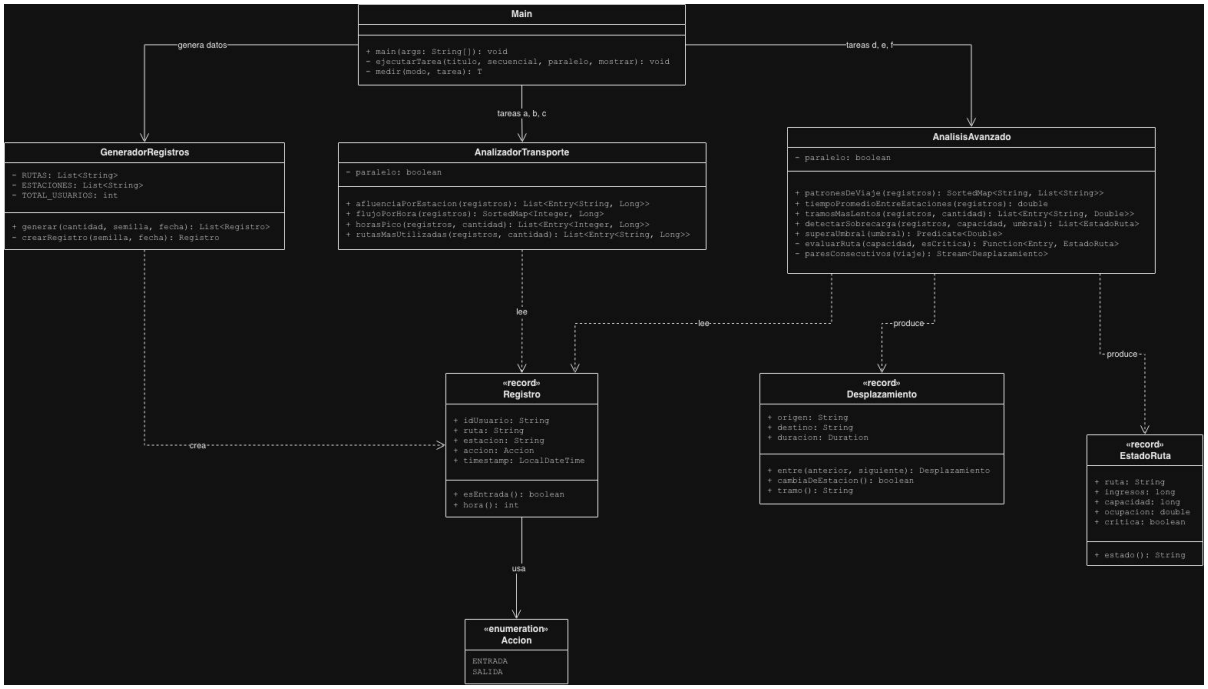
- Crear los registros con estructuras que no se pueden modificar (inmutables).
- Hacer las operaciones del análisis con funciones puras, lambdas y Streams.
- Comparar el procesamiento secuencial con el paralelo y revisar que den el mismo resultado.
- Subir la solución a un repositorio de GitHub, en la rama principal, trabajando en equipo.

3. Desarrollo

Organizamos el código en tres paquetes: *modelo* (los datos), *datos* (los datos simulados) y *servicio* (los cálculos), más la clase *Main*, que ejecuta todo. La Figura 1 muestra el diagrama de clases.

Figura 1

Diagrama de clases del módulo TecnoMovil Data



3.1 Modelo de datos

Cada registro es un paso de un usuario por una estación. Lo hicimos con un *record* llamado Registro, que tiene idUsuario, ruta, estación, acción y timestamp. Un *record* no se puede modificar después de crearlo, por eso es inmutable. Además revisa que ningún dato sea nulo y tiene dos métodos: esEntrada() y hora().

La acción es un *enum* llamado Accion, con los valores ENTRADA y SALIDA.

Para las tareas e y f agregamos otros dos *record*: Desplazamiento, que guarda el origen, el destino y el tiempo entre dos estaciones seguidas de un usuario, y EstadoRuta, que guarda la ruta, los ingresos, la capacidad, la ocupación y si la ruta es crítica.

Código 1

Clase Accion.java

```
package tecnomovil.modelo;

/**
 * Autor: Sara Gabriela Castro Mora
 * Fecha de creacion: 2026-11-01
 * Proposito: tipos de accion que registra un validador al paso de un usuario por una estacion.
 */
public enum Accion {

    /** El usuario ingresa a la estacion. */
    ENTRADA,

    /** El usuario valida para salir o para cambiar de ruta. */
    SALIDA
}
```

Código 2

Clase Registro.java

```
package tecnomovil.modelo;

import java.time.LocalDateTime;
import java.util.Objects;

/**
 * Autor: Sara Gabriela Castro Mora
 * Fecha de creacion: 2026-11-01
 * Proposito: registro inmutable de un paso de usuario por una estacion del sistema de
transporte.
 *
 * @param idUsuario identificador del usuario que valida su tarjeta
 * @param ruta      ruta en la que se produce el registro
 * @param estacion  estacion donde se produce el registro
 * @param accion    entrada o salida de la estacion
 * @param timestamp fecha y hora exacta del registro
 */
public record Registro(String idUsuario, String ruta, String estacion, Accion accion,
LocalDateTime timestamp) {

    /**
     * Valida que ningun campo del registro llegue nulo.
     */
    public Registro {
        Objects.requireNonNull(idUsuario, "idUsuario es obligatorio");
        Objects.requireNonNull(ruta, "ruta es obligatoria");
        Objects.requireNonNull(estacion, "estacion es obligatoria");
        Objects.requireNonNull(accion, "accion es obligatoria");
        Objects.requireNonNull(timestamp, "timestamp es obligatorio");
    }
```

```
    /**
     * Indica si el registro corresponde a un ingreso a la estacion.
     *
     * @return true cuando la accion es ENTRADA
     */
    public boolean esEntrada() {
        return accion == Accion.ENTRADA;
    }

    /**
     * Obtiene la hora del dia (0 a 23) en que ocurrio el registro.
     *
     * @return hora del timestamp
     */
    public int hora() {
        return timestamp.getHour();
    }
}
```

Código 3

Clase Desplazamiento.java

```
package tecnomovil.modelo;

import java.time.Duration;
import java.util.Objects;

/**
 * Fecha de creacion: 2026-11-04
 * Proposito: desplazamiento inmutable de un usuario entre dos estaciones consecutivas de su
viaje.
 *
 * @param origen   estacion del registro anterior
 * @param destino  estacion del registro siguiente
 * @param duracion tiempo transcurrido entre ambos registros
 */
public record Desplazamiento(String origen, String destino, Duration duracion) {

    /**
     * Valida que ningun campo llegue nulo y que la duracion no sea negativa.
     */
    public Desplazamiento {
        Objects.requireNonNull(origen, "origen es obligatorio");
        Objects.requireNonNull(destino, "destino es obligatorio");
        Objects.requireNonNull(duracion, "duracion es obligatoria");
        if (duracion.isNegative()) {
            throw new IllegalArgumentException("la duracion no puede ser negativa");
        }
    }

    /**
     * Funcion pura que construye el desplazamiento entre dos registros consecutivos de un
usuario.
     *
     * @param anterior  registro previo en orden cronologico
     * @param siguiente registro posterior en orden cronologico
     * @return desplazamiento de la estacion anterior a la siguiente
     */
    public static Desplazamiento entre(Registro anterior, Registro siguiente) {
        return new Desplazamiento(anterior.estacion(), siguiente.estacion(),
                Duration.between(anterior.timestamp(), siguiente.timestamp()));
    }
```

```
    /**
     * Indica si el usuario realmente cambio de estacion.
     *
     * @return true cuando origen y destino son distintos
     */
    public boolean cambiaDeEstacion() {
        return !origen.equals(destino);
    }

    /**
     * Nombre del tramo recorrido, por ejemplo "Portal Norte -> Universidad".
     *
     * @return tramo en formato origen -> destino
     */
    public String tramo() {
        return origen + " -> " + destino;
    }
}
```

Código 4

Clase EstadoRuta.java

```
package tecnomovil.modelo;

import java.util.Objects;

/**
 * Fecha de creacion: 2026-11-04
 * Proposito: resultado inmutable de evaluar la ocupacion diaria de una ruta frente a su
capacidad simulada.
 *
 * @param ruta      ruta evaluada
 * @param ingresos  total de ingresos de usuarios a la ruta en el dia
 * @param capacidad capacidad diaria simulada de la ruta
 * @param ocupacion ingresos divididos entre capacidad (1.0 equivale al 100 %)
 * @param critica   true cuando la ocupacion supera el umbral definido
 */
public record EstadoRuta(String ruta, long ingresos, long capacidad, double ocupacion, boolean
critica) {

    /**
     * Valida que la ruta exista y que los valores numericos sean coherentes.
     */
    public EstadoRuta {
        Objects.requireNonNull(ruta, "ruta es obligatoria");
        if (ingresos < 0 || capacidad <= 0) {
            throw new IllegalArgumentException("ingresos debe ser >= 0 y capacidad > 0");
        }
    }

    /**
     * Texto del estado para mostrar en reportes.
     *
     * @return "CRITICA" o "NORMAL"
     */
    public String estado() {
        return critica ? "CRITICA" : "NORMAL";
    }
}
```

3.2 Generador de datos simulados

La clase GeneradorRegistros crea una lista de 2.000.000 de registros simulados. Usa una semilla fija (2026), así que siempre salen los mismos datos. Cada registro se crea por separado, sin variables compartidas, y la mitad queda en horas pico para que se parezca a la realidad. La lista final se arma con toList(), que no se puede modificar.

Código 5

Clase GeneradorRegistros.java

```
package tecnomovil.datos;

import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.List;
import java.util.SplittableRandom;

import tecnomovil.modelo.Accion;
import tecnomovil.modelo.Registro;

/**
 * Autor: Sara Gabriela Castro Mora
 * Fecha de creacion: 2026-11-01
 * Proposito: generar un conjunto grande e inmutable de registros simulados del transporte
urbano.
 * La misma semilla siempre produce los mismos datos, lo que permite comparar resultados.
 */
public final class GeneradorRegistros {

    private static final List<String> RUTAS = List.of(
            "T01 Troncal Norte", "T02 Troncal Sur", "T03 Avenida Central", "A10 Alimentadora
Oriente",
            "A11 Alimentadora Occidente", "A12 Circular Universitaria", "E20 Expreso Aeropuerto",
"N30 Nocturna");

    private static final List<String> ESTACIONES = List.of(
            "Portal Norte", "Estacion Central", "Plaza Mayor", "Terminal Sur", "Parque
Industrial",
            "Universidad", "Hospital General", "Centro Administrativo", "Barrio Nuevo",
"Aeropuerto");

    private static final List<Integer> HORAS_PICO = List.of(6, 7, 8, 17, 18, 19);
    private static final int HORA_APERTURA = 5;
    private static final int HORA_CIERRE = 22;
    private static final int TOTAL_USUARIOS = 50_000;

    private GeneradorRegistros() {
    }

    /**
     * Genera una lista inmutable de registros simulados para una fecha dada.
     * Cada registro se construye a partir de su propia semilla, sin estado compartido.
     *
     * @param cantidad numero de registros a generar
     * @param semilla  semilla base para reproducir los mismos datos
     * @param fecha    dia al que pertenecen los registros
```

```
     * @return lista inmutable de registros
     */
    public static List<Registro> generar(int cantidad, long semilla, LocalDate fecha) {
        return new SplittableRandom(semilla)
                .longs(cantidad)
                .mapToObj(semillaRegistro -> crearRegistro(semillaRegistro, fecha))
                .toList();
    }

    /**
     * Construye un registro simulado a partir de una semilla. Concentra la mitad de los
     * registros en horas pico y favorece unas rutas y estaciones sobre otras.
     */
    private static Registro crearRegistro(long semilla, LocalDate fecha) {
        SplittableRandom aleatorio = new SplittableRandom(semilla);
        int hora = aleatorio.nextBoolean()
                ? HORAS_PICO.get(aleatorio.nextInt(HORAS_PICO.size()))
                : aleatorio.nextInt(HORA_APERTURA, HORA_CIERRE + 1);
        LocalDateTime timestamp = fecha.atTime(hora, aleatorio.nextInt(60),
aleatorio.nextInt(60));

        return new Registro(
                "U" + aleatorio.nextInt(1, TOTAL_USUARIOS + 1),
                elegirConSesgo(RUTAS, aleatorio),
                elegirConSesgo(ESTACIONES, aleatorio),
                aleatorio.nextBoolean() ? Accion.ENTRADA : Accion.SALIDA,
                timestamp);
    }

    /**
     * Elige un elemento de la lista dando mas probabilidad a los primeros,
     * para que los datos simulados tengan estaciones y rutas mas concurridas.
     */
    private static String elegirConSesgo(List<String> opciones, SplittableRandom aleatorio) {
        return opciones.get(Math.min(aleatorio.nextInt(opciones.size()),
aleatorio.nextInt(opciones.size())));
    }
}
```

3.3 Tareas a, b y c: clase AnalizadorTransporte

Las tareas a, b y c están en la clase AnalizadorTransporte. Sus métodos reciben la lista de registros, no la modifican y devuelven listas o mapas nuevos que tampoco se pueden modificar. La clase se puede usar con stream() o con parallelStream() y da el mismo resultado.

Tarea a: afluencia por estación

El método afluenciaPorEstacion cuenta cuántos usuarios entran a cada estación en el día. Primero filtra solo las entradas con Registro::esEntrada, después agrupa por estación con groupingBy y cuenta con counting().

Tarea b: horas pico

El método flujoPorHora agrupa todos los registros por hora del día. El método horasPico toma ese resultado, lo ordena de mayor a menor y deja las primeras N horas, que son las de más movimiento.

Tarea c: rutas más utilizadas

El método rutasMasUtilizadas funciona igual que la tarea a, pero agrupando por ruta, y deja las que tienen más ingresos.

Código 6

Clase AnalizadorTransporte.java

```
package tecnomovil.servicio;

import java.util.Collections;
import java.util.List;
import java.util.Map;
import java.util.SortedMap;
import java.util.TreeMap;
import java.util.stream.Collectors;
import java.util.stream.Stream;

import tecnomovil.modelo.Registro;

/**
 * Autor: Sara Gabriela Castro Mora
 * Fecha de creacion: 2026-11-01
 * Proposito: operaciones funcionales de analisis sobre los registros diarios del transporte.
 * Todas las operaciones son funciones puras: no modifican la lista recibida, no usan estado
 * compartido y devuelven colecciones inmutables, por eso funcionan igual en secuencial y en
paralelo.
 */
public final class AnalizadorTransporte {

    private final boolean paralelo;

    /**
     * Crea el analizador indicando si los pipelines se ejecutan con parallelStream().
     *
     * @param paralelo true para procesar en paralelo, false para procesar en secuencial
     */
    public AnalizadorTransporte(boolean paralelo) {
        this.paralelo = paralelo;
    }

    /**
     * a) Cuenta cuantos ingresos tiene cada estacion durante el dia.
     *
     * @param registros registros del dia
     * @return estaciones con su total de ingresos, de mayor a menor afluencia
     */
    public List<Map.Entry<String, Long>> afluenciaPorEstacion(List<Registro> registros) {
```

```
        Map<String, Long> ingresosPorEstacion = flujo(registros)
                .filter(Registro::esEntrada)
                .collect(Collectors.groupingBy(Registro::estacion, Collectors.counting()));

        return ordenarDeMayorAMenor(ingresosPorEstacion).toList();
    }

    /**
     * b) Agrupa todos los registros (entradas y salidas) por hora del dia.
     *
     * @param registros registros del dia
     * @return mapa inmutable ordenado por hora con el total de registros de cada hora
     */
    public SortedMap<Integer, Long> flujoPorHora(List<Registro> registros) {
        return flujo(registros)
                .collect(Collectors.collectingAndThen(
                        Collectors.groupingBy(Registro::hora, TreeMap::new,
Collectors.counting()),
                        Collections::unmodifiableSortedMap));
    }

    /**
     * b) Identifica las horas con mayor flujo de pasajeros.
     *
     * @param registros registros del dia
     * @param cantidad  numero de horas pico a devolver
     * @return horas con su total de registros, de mayor a menor flujo
     */
    public List<Map.Entry<Integer, Long>> horasPico(List<Registro> registros, int cantidad) {
        return ordenarDeMayorAMenor(flujoPorHora(registros))
                .limit(cantidad)
                .toList();
    }

    /**
     * c) Determina las rutas con mayor volumen de uso, medido por ingresos de usuarios.
     *
     * @param registros registros del dia
     * @param cantidad  numero de rutas a devolver
     * @return rutas con su total de ingresos, de mayor a menor uso
     */
    public List<Map.Entry<String, Long>> rutasMasUtilizadas(List<Registro> registros, int
cantidad) {
        Map<String, Long> ingresosPorRuta = flujo(registros)
                .filter(Registro::esEntrada)
                .collect(Collectors.groupingBy(Registro::ruta, Collectors.counting()));

        return ordenarDeMayorAMenor(ingresosPorRuta)
                .limit(cantidad)
                .toList();
    }

    /**
     * Abre el stream de registros en secuencial o en paralelo segun la configuracion.
     */
    private Stream<Registro> flujo(List<Registro> registros) {
        return paralelo ? registros.parallelStream() : registros.stream();
    }

    /**
     * Ordena un conteo de mayor a menor total; en empate ordena por clave para que el
     * resultado sea siempre el mismo. Copia cada par en una entrada inmutable.
     */
    private static <K extends Comparable<K>> Stream<Map.Entry<K, Long>>
ordenarDeMayorAMenor(Map<K, Long> conteos) {
        return conteos.entrySet().stream()
                .sorted(Map.Entry.<K, Long>comparingByValue().reversed()
```

```
                        .thenComparing(Map.Entry.comparingByKey()))
                .map(conteo -> Map.entry(conteo.getKey(), conteo.getValue()));
    }
}
```

3.4 Tareas d, e y f: clase AnalisisAvanzado

Las tareas d, e y f están en la clase AnalisisAvanzado. Igual que la otra clase, se puede usar de forma secuencial o paralela, no modifica la lista que recibe y devuelve colecciones inmutables.

Tarea d: patrones de viaje por usuario

El método patronesDeViaje agrupa los registros por usuario, ordena el viaje de cada uno por timestamp y lo convierte en la lista de estaciones que visitó con map(Registro::estacion). El resultado es un mapa ordenado por número de usuario.

Tarea e: tiempo promedio entre estaciones

El método tiempoPromedioEntreEstaciones toma el viaje ordenado de cada usuario y arma pares de registros seguidos (con IntStream.range, mapToObj y flatMap). Con cada par crea un Desplazamiento, descarta los que no cambian de estación y saca el promedio en minutos con average(). El método tramosMasLentos hace lo mismo, pero agrupa por tramo (origen -> destino) con groupingBy y averagingLong, y muestra los tramos que más se demoran.

Tarea f: detección de sobrecarga en rutas

El método detectarSobrecarga cuenta los ingresos de cada ruta y calcula la ocupación dividiéndolos entre una capacidad diaria simulada de 250.000. Si la ocupación pasa del 80 %, la ruta se marca como crítica. Para esto usamos dos funciones de orden superior: superaUmbral(umbral), que devuelve un Predicate, y evaluarRuta(capacidad, criterio), que devuelve la Function que convierte cada conteo en un EstadoRuta.

Código 7

Clase AnalisisAvanzado.java

```
package tecnomovil.servicio;

import java.util.Collections;
import java.util.Comparator;
import java.util.List;
import java.util.Map;
import java.util.SortedMap;
import java.util.TreeMap;
import java.util.function.Function;
import java.util.function.Predicate;
import java.util.stream.Collectors;
import java.util.stream.IntStream;
import java.util.stream.Stream;

import tecnomovil.modelo.Desplazamiento;
import tecnomovil.modelo.EstadoRuta;
import tecnomovil.modelo.Registro;

/**
 * Fecha de creacion: 2026-11-04
 * Proposito: tareas avanzadas d, e y f del modulo TecnoMovil Data (patrones de viaje, tiempos
entre
 * estaciones y sobrecarga de rutas). Igual que AnalizadorTransporte, todas las operaciones son
funciones
 * puras: no modifican la lista recibida, no usan estado compartido y devuelven colecciones
inmutables,
 * por eso dan el mismo resultado en secuencial y en paralelo.
 */
public final class AnalisisAvanzado {

    private static final double SEGUNDOS_POR_MINUTO = 60.0;

    /** Ordena los usuarios por su numero: U1, U2, ..., U10 en lugar de U1, U10, U2. */
    private static final Comparator<String> ORDEN_USUARIOS =
            Comparator.comparingInt(String::length).thenComparing(Comparator.naturalOrder());

    private static final Comparator<Registro> ORDEN_CRONOLOGICO =
Comparator.comparing(Registro::timestamp);

    private final boolean paralelo;

    /**
     * Crea el analizador indicando si los pipelines se ejecutan con parallelStream().
     *
     * @param paralelo true para procesar en paralelo, false para procesar en secuencial
     */
    public AnalisisAvanzado(boolean paralelo) {
        this.paralelo = paralelo;
    }

    /**
     * d) Genera, para cada usuario, el listado de estaciones que visito en orden cronologico.
     *
     * @param registros registros del dia
     * @return mapa inmutable usuario -> estaciones visitadas, ordenado por numero de usuario
     */
    public SortedMap<String, List<String>> patronesDeViaje(List<Registro> registros) {
        return viajesPorUsuario(registros).entrySet().stream()
                .collect(Collectors.collectingAndThen(
                        Collectors.toMap(
```

```
                                viaje -> viaje.getKey(),
                                viaje -> estacionesVisitadas(viaje.getValue()),
                                (primero, segundo) -> primero,
                                () -> new TreeMap<String, List<String>>(ORDEN_USUARIOS)),
                        Collections::unmodifiableSortedMap));
    }

    /**
     * e) Estima el tiempo promedio, en minutos, que tardan los usuarios en moverse entre
     * estaciones consecutivas de su viaje.
     *
     * @param registros registros del dia
     * @return promedio en minutos, o 0 si no hay desplazamientos
     */
    public double tiempoPromedioEntreEstaciones(List<Registro> registros) {
        return desplazamientos(registros)
                .mapToLong(desplazamiento -> desplazamiento.duracion().toSeconds())
                .average()
                .orElse(0)
                / SEGUNDOS_POR_MINUTO;
    }

    /**
     * e) Calcula el tiempo promedio por tramo (origen -> destino) y devuelve los tramos mas
lentos.
     *
     * @param registros registros del dia
     * @param cantidad  numero de tramos a devolver
     * @return tramos con su tiempo promedio en minutos, de mayor a menor
     */
    public List<Map.Entry<String, Double>> tramosMasLentos(List<Registro> registros, int
cantidad) {
        Map<String, Double> minutosPorTramo = desplazamientos(registros)
                .collect(Collectors.groupingBy(Desplazamiento::tramo,
                        Collectors.collectingAndThen(
                                Collectors.averagingLong(desplazamiento ->
desplazamiento.duracion().toSeconds()),
                                segundos -> segundos / SEGUNDOS_POR_MINUTO)));

        return minutosPorTramo.entrySet().stream()
                .sorted(Map.Entry.<String, Double>comparingByValue().reversed()
                        .thenComparing(Map.Entry.comparingByKey()))
                .limit(cantidad)
                .map(tramo -> Map.entry(tramo.getKey(), tramo.getValue()))
                .toList();
    }

    /**
     * f) Evalua la ocupacion diaria de cada ruta y marca como critica la que supera el umbral.
     *
     * @param registros       registros del dia
     * @param capacidadDiaria capacidad diaria simulada de cada ruta (ingresos)
     * @param umbral          fraccion de ocupacion a partir de la cual la ruta es critica (0.8 =
80 %)
     * @return estado de cada ruta, de mayor a menor ocupacion
     */
    public List<EstadoRuta> detectarSobrecarga(List<Registro> registros, long capacidadDiaria,
double umbral) {
        if (capacidadDiaria <= 0 || umbral <= 0) {
            throw new IllegalArgumentException("capacidadDiaria y umbral deben ser mayores que
cero");
        }
        return flujo(registros)
                .filter(Registro::esEntrada)
                .collect(Collectors.groupingBy(Registro::ruta, Collectors.counting()))
                .entrySet().stream()
                .map(evaluarRuta(capacidadDiaria, superaUmbral(umbral)))
                .sorted(Comparator.comparingDouble(EstadoRuta::ocupacion).reversed()
                        .thenComparing(EstadoRuta::ruta))
```

```
                .toList();
    }

    /**
     * f) Funcion de orden superior: construye el criterio que decide si una ocupacion es
critica.
     *
     * @param umbral fraccion de ocupacion limite
     * @return predicado que es verdadero cuando la ocupacion supera el umbral
     */
    public static Predicate<Double> superaUmbral(double umbral) {
        return ocupacion -> ocupacion > umbral;
    }

    /**
     * Funcion de orden superior: recibe la capacidad y el criterio de criticidad y devuelve
     * la funcion que transforma un conteo (ruta, ingresos) en el estado de la ruta.
     */
    private static Function<Map.Entry<String, Long>, EstadoRuta> evaluarRuta(long capacidad,
                                                                             Predicate<Double>
esCritica) {
        return conteo -> {
            double ocupacion = (double) conteo.getValue() / capacidad;
            return new EstadoRuta(conteo.getKey(), conteo.getValue(), capacidad, ocupacion,
                    esCritica.test(ocupacion));
        };
    }

    /**
     * Agrupa los registros por usuario, cada grupo ordenado cronologicamente. Lo usan las tareas
d y e.
     * groupingBy conserva el orden de aparicion incluso en paralelo, y el ordenamiento es
estable,
     * por lo que registros con el mismo timestamp mantienen el orden en que aparecen.
     */
    private Map<String, List<Registro>> viajesPorUsuario(List<Registro> registros) {
        return flujo(registros)
                .collect(Collectors.groupingBy(Registro::idUsuario,
                        Collectors.collectingAndThen(Collectors.toList(),
AnalisisAvanzado::ordenarCronologicamente)));
    }

    /**
     * Convierte los viajes de cada usuario en el flujo de desplazamientos entre estaciones
distintas.
     */
    private Stream<Desplazamiento> desplazamientos(List<Registro> registros) {
        Map<String, List<Registro>> viajes = viajesPorUsuario(registros);
        Stream<List<Registro>> flujoViajes = paralelo ? viajes.values().parallelStream() :
viajes.values().stream();
        return flujoViajes.flatMap(AnalisisAvanzado::paresConsecutivos);
    }

    /**
     * Funcion pura: recorre un viaje ordenado y produce un desplazamiento por cada par de
     * registros consecutivos en estaciones diferentes.
     */
    private static Stream<Desplazamiento> paresConsecutivos(List<Registro> viaje) {
        return IntStream.range(1, viaje.size())
                .mapToObj(indice -> Desplazamiento.entre(viaje.get(indice - 1),
viaje.get(indice)))
                .filter(Desplazamiento::cambiaDeEstacion);
    }

    /**
     * Funcion pura: devuelve una copia inmutable de los registros ordenada por timestamp.
     */
    private static List<Registro> ordenarCronologicamente(List<Registro> registros) {
```

```
        return registros.stream().sorted(ORDEN_CRONOLOGICO).toList();
    }

    /**
     * Funcion pura: extrae la secuencia de estaciones de un viaje ya ordenado.
     */
    private static List<String> estacionesVisitadas(List<Registro> viaje) {
        return viaje.stream().map(Registro::estacion).toList();
    }

    /**
     * Abre el stream de registros en secuencial o en paralelo segun la configuracion.
     */
    private Stream<Registro> flujo(List<Registro> registros) {
        return paralelo ? registros.parallelStream() : registros.stream();
    }
}
```

3.5 Clase principal Main

Main es la clase que arranca el programa. Genera los 2.000.000 de registros, ejecuta las tareas de la a a la f de forma secuencial y paralela, mide cuánto se demora cada una y muestra los resultados en consola. También compara los dos resultados para confirmar que son iguales. El método ejecutarTarea recibe las tareas (Supplier) y la forma de mostrarlas (Consumer) como parámetros, así se usa el mismo código para todas.

Código 8

Clase Main.java

```
package tecnomovil;

import java.time.LocalDate;
import java.util.List;
import java.util.Map;
import java.util.SortedMap;
import java.util.function.Consumer;
import java.util.function.Supplier;

import tecnomovil.datos.GeneradorRegistros;
import tecnomovil.modelo.EstadoRuta;
import tecnomovil.modelo.Registro;
import tecnomovil.servicio.AnalisisAvanzado;
import tecnomovil.servicio.AnalizadorTransporte;

/**
 * Autor: Sara Gabriela Castro Mora
 * Fecha de creacion: 2026-11-01
 * Actualizacion: 2026-11-04 - se integran las tareas d, e y f (AnalisisAvanzado).
 * Proposito: punto de entrada del modulo TecnoMovil Data. Genera los datos simulados, ejecuta
 * las tareas a a f en secuencial y en paralelo, y verifica que ambos modos den el mismo
resultado.
 */
public final class Main {
```

```
    private static final int TOTAL_REGISTROS = 2_000_000;
    private static final long SEMILLA = 2026L;
    private static final int TOP = 5;
    private static final int USUARIOS_DE_EJEMPLO = 3;
    private static final int ESTACIONES_DE_EJEMPLO = 8;
    private static final long CAPACIDAD_DIARIA_POR_RUTA = 250_000L;
    private static final double UMBRAL_CRITICO = 0.80;

    private Main() {
    }

    /**
     * Ejecuta el analisis completo y muestra los resultados en consola.
     *
     * @param args no se usan
     */
    public static void main(String[] args) {
        List<Registro> registros = GeneradorRegistros.generar(TOTAL_REGISTROS, SEMILLA,
LocalDate.of(2026, 11, 1));
        System.out.printf("Registros simulados: %,d%n%n", registros.size());

        AnalizadorTransporte secuencial = new AnalizadorTransporte(false);
        AnalizadorTransporte paralelo = new AnalizadorTransporte(true);
        AnalisisAvanzado avanzadoSecuencial = new AnalisisAvanzado(false);
        AnalisisAvanzado avanzadoParalelo = new AnalisisAvanzado(true);

        ejecutarTarea("a) Afluencia por estacion (ingresos)",
                () -> secuencial.afluenciaPorEstacion(registros),
                () -> paralelo.afluenciaPorEstacion(registros),
                Main::mostrarConteos);

        ejecutarTarea("b) Horas pico (top " + TOP + ", entradas y salidas)",
                () -> secuencial.horasPico(registros, TOP),
                () -> paralelo.horasPico(registros, TOP),
                Main::mostrarConteos);

        ejecutarTarea("c) Rutas mas utilizadas (top " + TOP + ", ingresos)",
                () -> secuencial.rutasMasUtilizadas(registros, TOP),
                () -> paralelo.rutasMasUtilizadas(registros, TOP),
                Main::mostrarConteos);

        ejecutarTarea("d) Patrones de viaje por usuario (primeros " + USUARIOS_DE_EJEMPLO + "
usuarios)",
                () -> avanzadoSecuencial.patronesDeViaje(registros),
                () -> avanzadoParalelo.patronesDeViaje(registros),
                Main::mostrarPatrones);

        ejecutarTarea("e) Tiempo promedio entre estaciones consecutivas",
                () -> avanzadoSecuencial.tiempoPromedioEntreEstaciones(registros),
                () -> avanzadoParalelo.tiempoPromedioEntreEstaciones(registros),
                minutos -> System.out.printf("  Promedio general: %.2f minutos%n", minutos));

        ejecutarTarea("e) Tramos con mayor tiempo promedio (top " + TOP + ")",
                () -> avanzadoSecuencial.tramosMasLentos(registros, TOP),
                () -> avanzadoParalelo.tramosMasLentos(registros, TOP),
                Main::mostrarMinutos);

        ejecutarTarea(String.format("f) Sobrecarga de rutas (capacidad %,d ingresos/dia, umbral
%.0f %%)",
                        CAPACIDAD_DIARIA_POR_RUTA, UMBRAL_CRITICO * 100),
                () -> avanzadoSecuencial.detectarSobrecarga(registros, CAPACIDAD_DIARIA_POR_RUTA,
UMBRAL_CRITICO),
                () -> avanzadoParalelo.detectarSobrecarga(registros, CAPACIDAD_DIARIA_POR_RUTA,
UMBRAL_CRITICO),
```

```
                Main::mostrarEstadosRuta);
    }


    /**
     * Ejecuta una tarea en ambos modos, muestra su resultado e indica si secuencial y paralelo
coinciden.
     * Es una funcion de orden superior: recibe las tareas y la forma de mostrarlas como
funciones.
     */
    private static <T> void ejecutarTarea(String titulo, Supplier<T> tareaSecuencial, Supplier<T>
tareaParalela,
                                          Consumer<T> mostrar) {
        System.out.println(titulo);
        T secuencial = medir("secuencial", tareaSecuencial);
        T paralelo = medir("paralelo", tareaParalela);
        mostrar.accept(secuencial);
        System.out.println("  Secuencial y paralelo coinciden: " + secuencial.equals(paralelo));
        System.out.println();
    }


    /**
     * Ejecuta una tarea, imprime cuanto tardo y devuelve su resultado.
     */
    private static <T> T medir(String modo, Supplier<T> tarea) {
        long inicio = System.nanoTime();
        T resultado = tarea.get();
        System.out.printf("  [%s] %d ms%n", modo, (System.nanoTime() - inicio) / 1_000_000);
        return resultado;
    }


    private static <K> void mostrarConteos(List<Map.Entry<K, Long>> conteos) {
        conteos.forEach(conteo -> System.out.printf("  %-28s %,10d%n", conteo.getKey(),
conteo.getValue()));
    }


    private static void mostrarMinutos(List<Map.Entry<String, Double>> tramos) {
        tramos.forEach(tramo -> System.out.printf("  %-46s %8.2f min%n", tramo.getKey(),
tramo.getValue()));
    }


    private static void mostrarPatrones(SortedMap<String, List<String>> patrones) {
        System.out.printf("  Usuarios con viajes: %,d%n", patrones.size());
        patrones.entrySet().stream()
                .limit(USUARIOS_DE_EJEMPLO)
                .forEach(patron -> System.out.printf("  %-7s (%d estaciones) %s%s%n",
                        patron.getKey(),
                        patron.getValue().size(),
                        String.join(" > ", patron.getValue().subList(0,
                                Math.min(ESTACIONES_DE_EJEMPLO, patron.getValue().size()))),
                        patron.getValue().size() > ESTACIONES_DE_EJEMPLO ? " > ..." : ""));
    }


    private static void mostrarEstadosRuta(List<EstadoRuta> estados) {
        estados.forEach(estado -> System.out.printf("  %-28s %,10d  %6.1f %%  %s%n",
                estado.ruta(), estado.ingresos(), estado.ocupacion() * 100, estado.estado()));
        System.out.printf("  Rutas criticas: %d de %d%n",
                estados.stream().filter(EstadoRuta::critica).count(), estados.size());
    }
}
```

3.6 Conceptos de programación funcional aplicados

Funciones puras

Una función pura siempre da el mismo resultado con los mismos datos y no cambia nada por fuera. Todos los métodos de AnalizadorTransporte y AnalisisAvanzado lo hacen: reciben la lista, la procesan y devuelven un resultado nuevo, sin modificar variables externas. También son puros esEntrada() y hora() de Registro. Lo único que escribe en pantalla es Main, que quedó separado de los cálculos.

Inmutabilidad

Ningún dato se modifica después de crearse. Registro, Desplazamiento y EstadoRuta son *record*, las listas se crean con List.of() y toList(), y el mapa de horas se entrega con Collections.unmodifiableSortedMap. Así no hay condiciones de carrera, que era el problema del sistema anterior.

Funciones de orden superior

Son funciones que reciben otras funciones o las devuelven. Las usamos en filter, map, sorted y collect, que reciben funciones, y en ejecutarTarea de Main, que recibe dos Supplier y un Consumer. También superaUmbral y evaluarRuta devuelven funciones. Además combinamos comparadores con Map.Entry.comparingByValue().reversed().thenComparing(...).

Expresiones lambda

Son funciones cortas sin nombre. Algunos ejemplos del código son semillaRegistro -> crearRegistro(semillaRegistro, fecha) en el generador, ocupacion -> ocupacion > umbral en superaUmbral y () -> secuencial.afluenciaPorEstacion(registros) en Main. Cuando la lambda solo llama a un método usamos referencias a métodos, como Registro::esEntrada o Registro::estacion.

Streams

Con los Streams se dice qué se quiere calcular, sin recorrer la lista con ciclos. Cada tarea es una cadena de operaciones: primero las intermedias (filter, sorted, limit, map, flatMap) y al final una terminal (collect o toList). Como cada elemento se procesa por separado, la misma cadena se puede correr con parallelStream() sin cambiar el código.

3.7 Evidencia de ejecución

Compilamos y ejecutamos el programa con los 2.000.000 de registros simulados. Las Figuras 2 y 3 muestran la salida en consola de las tareas a, b, c, d, e y f, con el tiempo de cada una en modo secuencial y paralelo.

Figura 2

Compilación y salida en consola de las tareas a, b, c y d

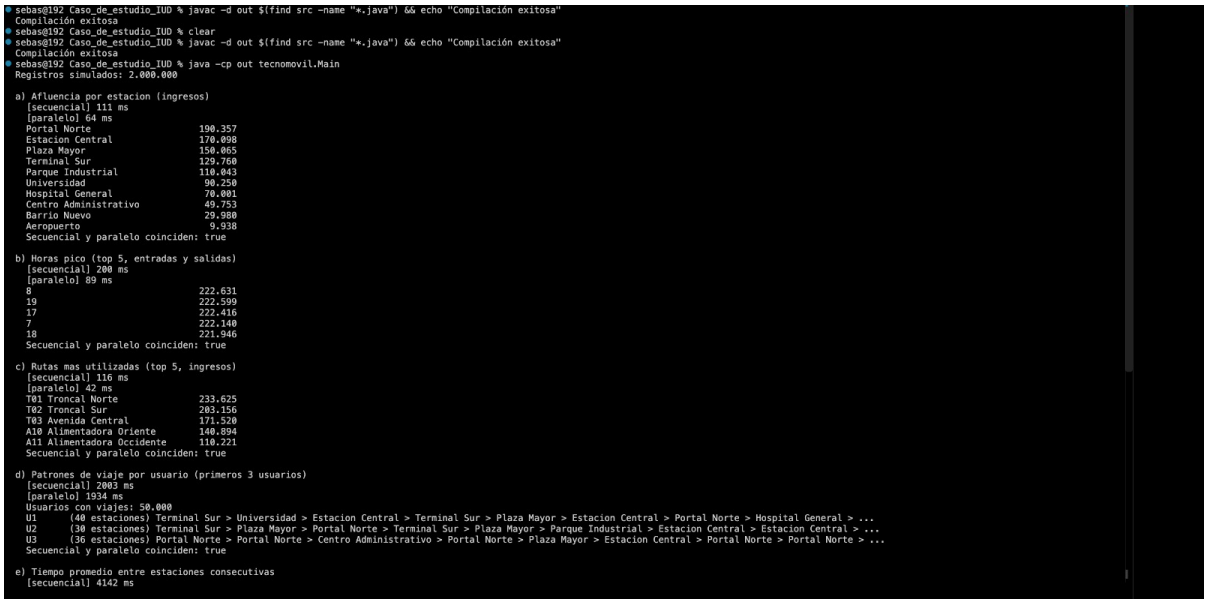
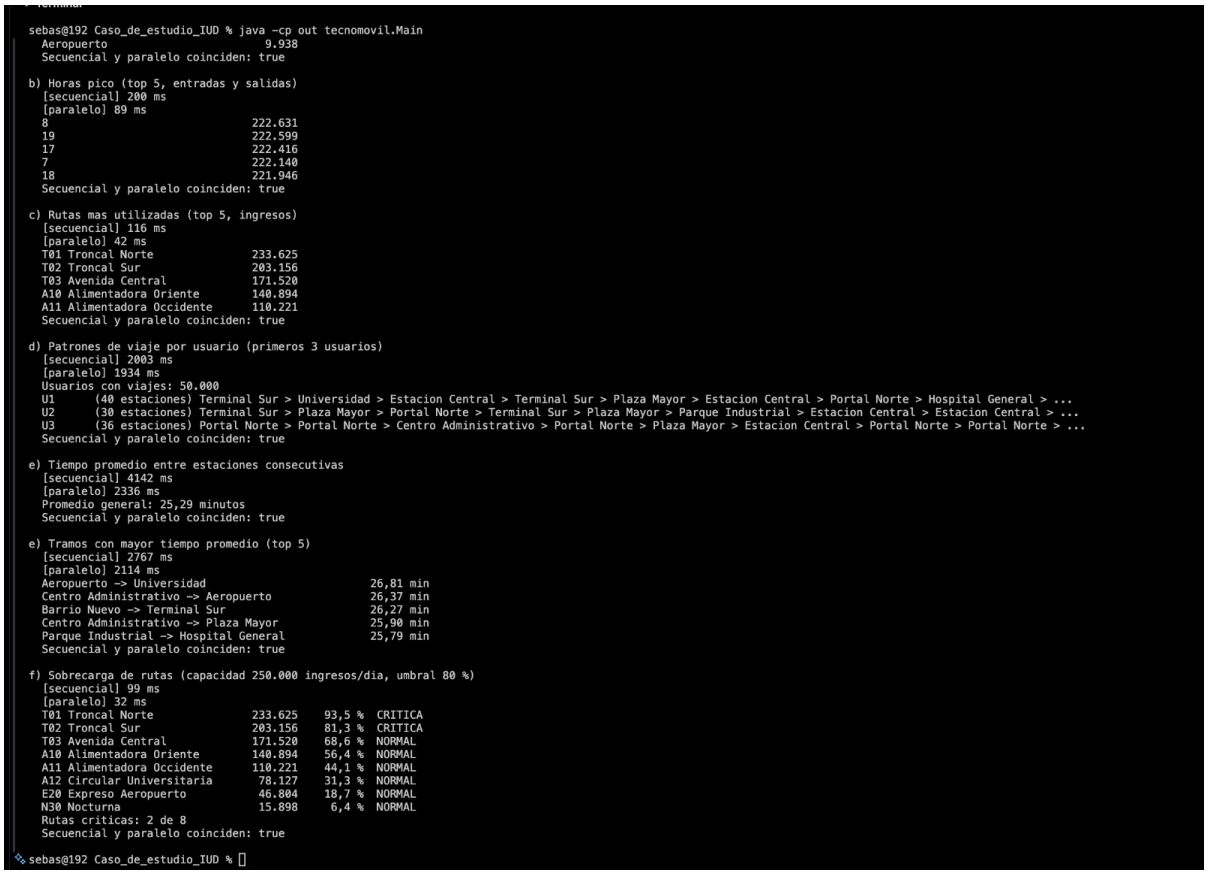


Figura 3

Salida en consola de las tareas b a f



En todas las tareas el modo paralelo se demoró menos que el secuencial. Por ejemplo, la afluencia por estación pasó de 111 ms a 64 ms, y el tiempo promedio entre estaciones pasó de 4142 ms a 2336 ms. En todas aparece "Secuencial y paralelo coinciden: true", es decir, los dos modos dan el mismo resultado.

En los resultados, Portal Norte fue la estación con más ingresos (190.357), la hora 8 fue la de mayor flujo y la ruta T01 Troncal Norte fue la más usada. Además, 2 de las 8 rutas pasaron el umbral del 80 % y quedaron como críticas: T01 Troncal Norte (93,5 %) y T02 Troncal Sur (81,3 %). El tiempo promedio entre estaciones fue de 25,29 minutos.

3.8 Repositorio y trabajo colaborativo

El proyecto está en el repositorio de GitHub Caso_de_estudio_IUD. Cada uno trabajó en su rama (sara y rama-b) para no dañar el trabajo del otro, y después unimos los cambios a la rama principal (main) con *pull requests*. El enlace del repositorio está en la sección 5.

4. Conclusiones

Logramos hacer en Java un módulo que procesa 2.000.000 de registros simulados de transporte usando solo Streams, lambdas, funciones puras y colecciones inmutables, sin frameworks externos. Con esto cumplimos el objetivo de trabajar con muchos datos de forma declarativa.

El enfoque funcional ayuda con los problemas del sistema anterior. Como no se modifican datos compartidos, no hay condiciones de carrera, y cada tarea quedó en pocas líneas, más fácil de leer que un recorrido con ciclos y contadores.

Como las funciones son puras y los datos no cambian, el mismo análisis se puede correr con stream() o con parallelStream() y da el mismo resultado, como lo comprobamos en Main. En la ejecución, el modo paralelo fue más rápido en todas las tareas.

Las tareas d, e y f mostraron que el mismo enfoque sirve para cálculos más complejos, como armar los viajes de cada usuario, medir el tiempo entre estaciones y detectar rutas críticas con funciones de orden superior.

Por último, trabajar con GitHub, con ramas separadas y pull requests, nos permitió dividir las tareas y juntar el código en main sin perder el trabajo de ninguno.

5. Enlaces de entrega

Repositorio del grupo (Sebastián Giraldo Anaya y Sara Gabriela Castro Mora):

https://github.com/Sebasg006/Caso_de_estudio_IUD

6. Referencias

Institución Universitaria Digital de Antioquia. (2026). *Caso de estudio Unidad 2: procesamiento funcional de datos para un sistema de gestión de transporte urbano*. Programación Orientada a Objetos II.

Oracle. (s. f.). *Package java.util.stream*. Java Platform, Standard Edition API Specification. https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/stream/package-summary.html